LAWRENCE TECHNOLOGICAL UNIVERSITY

Big Data Analytics (MCS 5343)

Project on

Social Media Sentiment and Trend Analysis

Project Team

Ishaan Pathak

Ali Arshad Mohammed

Rahul Potharaju

1. Abstract

The exponential growth of social media platforms has generated massive volumes of high-velocity, unstructured data that reflect public opinions, behaviors, and emerging trends. Analyzing this data at scale requires analytics workflows that can process large corpora efficiently while preserving useful semantic information.

Traditional text-processing approaches struggle with noisy, short-form social media language and often fail to capture contextual meaning. There is therefore a strong need for scalable big data analytics frameworks that combine distributed preprocessing with modern embedding-based language models.

This project designs and implements a scalable social media analytics workflow for sentiment and trend analysis using Apache Spark for large-scale preprocessing and transformer-based text representations for semantic modeling.

The implemented pipeline uses the Sentiment140 dataset, performs Spark-based data cleaning and time-feature extraction, generates contextual tweet embeddings with a pretrained DistilBERT model, clusters the embedding space with MiniBatchKMeans, and visualizes the resulting structure using Incremental PCA.

Experimental results on 1,583,571 cleaned tweets show that the proposed workflow can identify broad semantic usage patterns across the corpus while also enabling post-cluster sentiment interpretation. The final analysis revealed five interpretable clusters, including two negative-leaning clusters and three positive-leaning clusters.

Overall, the project demonstrates the effectiveness of integrating big data preprocessing with transformer embeddings for large-scale social media analytics. The framework has practical relevance for brand monitoring, public-opinion analysis, and large-scale exploratory trend discovery.

Keywords: Social Media Analytics, Big Data, Apache Spark, Deep Learning, Transformer Embeddings, Sentiment Analysis, Trend Detection, Unsupervised Clustering.

2. Introduction

2.1 Background of the Study

The rapid expansion of digital technologies has transformed the way data is generated, stored, and analyzed. Social media platforms produce massive volumes of user-generated content in the form of posts, comments, hashtags, images, and interactions. This data embodies the five key characteristics of Big Data: **Volume** (large-scale datasets), **Velocity** (continuous real-time data streams), **Variety** (text, images, videos, metadata), **Veracity** (data uncertainty and noise), and **Value** (actionable insights derived from analysis). Traditional relational databases and centralized processing systems were not designed to handle such scale and complexity efficiently.

Historically, data processing relied on batch-based systems where data was collected, stored, and analyzed periodically. Technologies such as **Apache Hadoop** established the foundation for distributed batch

processing of large datasets. However, modern applications require near realtime insight generation rather than delayed batch analysis. This need has led to the adoption of distributed and streaming technologies such as **Apache Spark** for fast in-memory computation, **Apache Kafka** for real-time data ingestion, and **Apache Flink** for low-latency stream processing.

In the context of social media analytics, scalable processing enables organizations and researchers to extract useful insights from very large collections of posts, detect recurring discussion patterns, and monitor shifts in sentiment over time. Such workflows also provide a practical foundation for future near-real-time analytics systems.

2.2. Problem Statement

Conventional small-scale or purely keyword-based approaches are often insufficient for analyzing large social media corpora composed of short, noisy, and semantically diverse posts. As dataset size increases, it becomes challenging to preserve both scalability and contextual understanding.

The primary challenges include:

- Handling very large volumes of short, noisy text data
- Ensuring scalability as dataset size grows
- Capturing semantic meaning beyond simple keyword counts
- Producing interpretable trend groupings from unstructured text
- Integrating sentiment information with large-scale semantic analysis

There is a clear real-world need for scalable social media analytics in areas such as brand monitoring, public opinion tracking, crisis communication analysis, and digital marketing. Without efficient large-scale analytics, organizations risk overlooking broad discussion patterns and sentiment shifts.

2.3. Aim and Objectives of the Project

Aim

To design and implement a scalable big data analytics system for large-scale social media text processing, semantic trend detection, and sentiment-oriented analysis using distributed preprocessing and transformer-based embeddings.

Objectives

- To build a scalable data-loading and preprocessing pipeline for social media text.
- To perform distributed analytics and exploratory trend analysis using Apache Spark.
- To generate transformer-based embeddings for semantic representation of tweets.

- To group tweets into interpretable clusters using unsupervised learning.
- To analyze sentiment composition within the discovered semantic clusters.
- To visualize large-scale semantic structure using dimensionality reduction.

2. 4. Significance of the Study

Scalable social media analytics plays a crucial role in modern decision-making environments. For businesses, it supports market intelligence and brand reputation monitoring across large volumes of user-generated text. In public administration, journalism, and social research, large-scale sentiment and trend analysis can help reveal broad public reactions and recurring discussion patterns.

Academically, this project contributes to understanding how distributed big data frameworks can be combined with transformer-based natural language processing for scalable trend analysis. Technologically, it demonstrates a practical workflow for large-scale text preprocessing, semantic modeling, clustering, and visualization.

2. 5. Scope of the Project

This project focuses on:

- Large-scale social media data preprocessing and analysis
- Distributed processing using Apache Spark
- Transformer-based embeddings for semantic text representation
- Unsupervised clustering, sentiment composition analysis, and visualization

The project does not implement a fully deployed live streaming architecture or end-to-end online inference service. Instead, it uses an offline benchmark dataset and demonstrates a scalable analytics workflow that can serve as a foundation for future streaming extensions.

2.6. Research Questions

This study is guided by the following research questions:

1. How can a large social media text corpus be processed efficiently using distributed big data tools?
2. How effectively do transformer-based embeddings capture semantic similarity in short social media posts?
3. What kinds of interpretable semantic clusters emerge from large-scale tweet embeddings, and how do they relate to sentiment composition?

4. How useful is dimensionality reduction for visualizing large-scale tweet clusters?

2.7. Brief Methodology Overview

The project implements a scalable analytics workflow consisting of dataset loading, Spark-based preprocessing, exploratory data analysis, contextual embedding generation, unsupervised clustering, post-cluster sentiment interpretation, and visualization. The final evaluation emphasizes cluster distribution, semantic interpretation, sentiment composition, and PCA-based structure rather than supervised classification accuracy.

3. Literature Review

3.1 Introduction

The rapid growth of social media platforms has led to the generation of massive volumes of usergenerated content in real time. Platforms such as Twitter (X), Facebook, Instagram, and Reddit produce continuous streams of textual, visual, and interaction-based data that reflect public opinions, behaviors, and emerging social phenomena. This section reviews existing research related to big data processing, social media analytics, trend detection, sentiment analysis, and deep learning–based text representations. It also examines distributed streaming architectures used for large-scale real-time analytics.

3.2 Big Data Characteristics and Distributed Processing

Big data is commonly characterized by the five V's: Volume, Velocity, Variety, Veracity, and Value. Social media data strongly exhibits all these characteristics due to its continuous generation, heterogeneous format, and high uncertainty. Traditional centralized databases were designed for structured data and periodic batch analysis, making them inefficient for large-scale, real-time social streams.

The introduction of distributed computing frameworks significantly improved large-scale data processing. Batch-oriented systems such as **Apache Hadoop** enabled scalable storage and processing of massive datasets using distributed file systems and parallel computation. However, batch frameworks process data in intervals and are not optimized for low-latency decision-making.

To address performance limitations, modern distributed engines such as **Apache Spark** introduced in-memory computation to accelerate data analytics. Furthermore, streaming platforms like **Apache Kafka** and real-time processing engines such as **Apache Flink** allow continuous eventdriven processing, making them suitable for real-time social media analytics.

3.3 Social Media Trend Detection

Trend detection is a central research area in social media analytics. Early approaches relied on frequency-based keyword tracking and burst detection algorithms to identify rapidly increasing terms or hashtags.

While effective in identifying simple popularity spikes, these methods often fail to capture semantic similarity or contextual meaning.

Subsequent research introduced clustering and topic modeling approaches, where posts are grouped based on content similarity to detect emerging discussion themes. Latent Dirichlet Allocation (LDA) became a widely used probabilistic topic modeling method; however, LDA performs less effectively on short texts typical of social media posts.

Recent studies emphasize embedding-based clustering methods, where textual content is converted into dense vector representations before grouping. This approach enables detection of semantically related discussions even when exact keywords differ. Embedding-driven trend detection has shown improved coherence and robustness compared to purely frequency-based techniques.

3.4 Sentiment Analysis in Social Media

Sentiment analysis aims to determine the emotional polarity or attitude expressed in textual content. Traditional approaches relied on lexicon-based methods and classical machine learning models such as Naïve Bayes, Support Vector Machines (SVM), and Logistic Regression using handcrafted features.

With advancements in deep learning, neural network architectures such as Recurrent Neural Networks (RNNs), Long Short-Term Memory (LSTM) networks, and Convolutional Neural Networks (CNNs) significantly improved sentiment classification accuracy. More recently, transformer-based architectures such as BERT and RoBERTa have become dominant due to their contextual understanding capabilities.

These transformer models generate contextual embeddings that capture the meaning of words depending on surrounding text. Such embeddings are particularly useful for social media analysis where language is informal, abbreviated, and context-dependent. Research consistently demonstrates that transformer-based models outperform traditional methods in sentiment classification tasks.

3.5 Text Embeddings and Representation Learning

Text representation plays a critical role in natural language processing tasks. Earlier vectorization techniques such as Bag-of-Words and TF-IDF treat words independently and produce sparse vectors, limiting semantic understanding.

The introduction of dense word embeddings such as Word2Vec and GloVe marked a major advancement in representation learning. These embeddings capture semantic similarity between words in continuous vector space. However, static embeddings assign the same vector to a word regardless of context.

Contextual embeddings generated by transformer models overcome this limitation by dynamically adjusting word representations based on surrounding words. Sentence-level embeddings further enhance clustering and topic detection by representing entire posts as single dense vectors. Embedding-based approaches are now widely used in sentiment analysis, topic modeling, and trend detection.

3.6 Real-Time Streaming Architectures for Social Media Analytics

Modern social media analytics systems are increasingly designed using streaming architectures. These systems typically follow a pipeline structure:

- Data ingestion from APIs or streaming platforms
- Real-time preprocessing and filtering
- Distributed processing and aggregation
- Model inference (sentiment, topic, trend detection)
- Storage and visualization

Streaming architectures reduce latency and enable continuous insight generation. Research highlights the importance of fault tolerance, horizontal scalability, and throughput optimization in such systems. Real-time analytics frameworks are particularly critical in domains requiring immediate response, such as crisis management, brand monitoring, and public opinion tracking.

3.7 Identified Research Gaps

Although significant progress has been made in social media analytics, several challenges remain:

- Many systems focus either on distributed scalability or model accuracy but do not effectively integrate both.
- Embedding-based deep learning models improve semantic understanding but increase computational cost in real-time systems.
- Handling noisy, short, and rapidly evolving social media text remains complex.
- Concept drift, where discussion topics shift over time, requires continuous model adaptation.

These challenges highlight the need for integrated frameworks that combine scalable distributed streaming architectures with embedding-based deep learning models for robust real-time trend analysis.

3.8 Summary

The literature indicates a clear evolution from batch-based data processing toward real-time streaming analytics. At the same time, natural language processing has progressed from sparse text representations to contextual embedding-based deep learning models. Integrating these two advancements provides a strong foundation for scalable and accurate social media trend analysis systems.

Building upon these developments, this project applies distributed preprocessing and embedding-driven semantic modeling to large-scale social media analysis, with a focus on practical trend discovery and sentiment interpretation on an offline benchmark dataset.

4. Research Design and Methodology

4.1 Research Design

This study adopts an applied experimental research design to analyze social media data using big data analytics and deep learning techniques. The goal is to develop a scalable workflow capable of processing a large corpus of social media text and identifying meaningful semantic trends and sentiment patterns.

The research design integrates distributed preprocessing with transformer-based language modeling. Social media text is passed through data loading, cleaning, feature engineering, embedding generation, unsupervised clustering, and visualization stages. This design reflects a practical big data analytics pipeline for large-scale text understanding.

The experimental design evaluates the system primarily through the quality and interpretability of the resulting clusters, their sentiment composition, and the ability of the workflow to scale across the full dataset through chunked processing.

4.2 System Architecture

The proposed system follows a scalable big data processing architecture designed to handle a large social media text corpus. The architecture consists of multiple components responsible for dataset loading, distributed processing, semantic modeling, clustering, and result visualization.
The system architecture includes the following stages:

1. Dataset Loading Layer
   The social media dataset is loaded from storage into the analytics environment. In this project, the Sentiment140 benchmark dataset was used as the primary data source.
2. Distributed Processing Layer
   The collected data is processed using Apache Spark. This layer performs schema assignment, text cleaning, duplicate removal, timestamp conversion, label mapping, and generation of time-based features for downstream analysis.
3. Feature Extraction Layer
   Textual data is converted into contextual numerical representations using a pretrained transformer model. These dense embeddings capture semantic relationships more effectively than sparse keyword-based representations.
4. Clustering and Analysis Layer
   The embedding vectors are analyzed using MiniBatchKMeans to identify semantic groups of tweets. Post-cluster sentiment composition is then examined to understand how positive and negative tweets are distributed within each semantic cluster.
5. Storage and Visualization Layer
   The clustered outputs, summary tables, and visualizations are saved for interpretation. Incremental PCA is used to project the high-dimensional embedding space into two dimensions for exploratory visualization.

This modular architecture enables scalable handling of large volumes of social media text while keeping the workflow interpretable and computationally practical.

4.3 Dataset Description

The dataset used in this study was the Sentiment140 dataset, a large social media benchmark containing tweets and associated sentiment labels. After preprocessing, the final corpus used for semantic analysis contained 1,583,571 tweets.
The dataset includes the following attributes:

- Tweet text
- Timestamp
- User identifier
- Binary sentiment label
- Record identifier

The dataset was preprocessed to remove noise such as URLs, user mentions, duplicate records, and irregular characters. Timestamp values were converted into structured date features, and the original Sentiment140 labels were mapped to a binary 0/1 representation for downstream sentiment interpretation.

4.4 Data Preprocessing

Before performing analytics, the raw social media data underwent several preprocessing steps to improve quality and consistency. The preprocessing pipeline includes:

1. Data Cleaning
   Removal of URLs, user mentions, punctuation noise, and unnecessary special characters while preserving transformer-friendly text structure.
2. Duplicate and Null Handling
   Removal of duplicate records and rows with missing or unusable text content.
3. Timestamp Formatting and Feature Extraction
   Conversion of raw date strings into timestamp format and derivation of year, month, day, and hour features for temporal analysis.
4. Label Mapping
   Conversion of the original Sentiment140 labels into a binary format where 0 represents negative sentiment and 1 represents positive sentiment.
5. Vectorization Preparation
   Retention of cleaned text in a form suitable for transformer tokenization and embedding generation.

These preprocessing steps improved the reliability of downstream exploratory analysis and semantic modeling.

4.5 Deep Learning Models and Embeddings

To analyze social media text effectively, the system employs transformer-based contextual embeddings. Each tweet is converted into a dense vector representation that captures semantic relationships and contextual meaning.

The final implementation used the following modeling components:

- Pretrained DistilBERT embeddings used as a frozen feature extractor
- MiniBatchKMeans clustering for scalable semantic grouping

This modeling strategy allows the system to represent tweet meaning without training a transformer model from scratch. Mean pooling was applied to token-level outputs to obtain tweet-level embeddings, and the resulting vectors were clustered to identify broad semantic patterns.

Because the final trend-discovery stage was unsupervised, the model was not optimized as a supervised sentiment classifier. Instead, sentiment labels were analyzed after clustering to interpret whether each semantic group leaned more negative or more positive.

4.6 Performance Evaluation

The proposed system was evaluated primarily using analysis-oriented criteria rather than supervised classification metrics.

Evaluation Criteria

- Cluster distribution across the full dataset
- Top words and sample tweets for semantic interpretation
- Sentiment composition within each cluster
- PCA-based visualization of cluster structure

These criteria help evaluate both the scalability of the workflow and the interpretability of the discovered semantic patterns.

4.7 Methodology Workflow

The complete workflow of the proposed system follows the sequence below:

Sentiment140 Dataset
↓
Spark Loading and Preprocessing
↓
Exploratory Data Analysis
↓
DistilBERT Embedding Generation
↓
MiniBatchKMeans Clustering
↓
Cluster-wise Sentiment Interpretation
↓
PCA Visualization and Insight Generation

4.8 Experimental Setup & Cluster Distribution

The experimental evaluation was conducted on the cleaned Sentiment140 dataset after Spark-based preprocessing. The final corpus used for semantic analysis contained 1,583,571 tweets with binary sentiment labels mapped to 0 (negative) and 1 (positive).

A pretrained DistilBERT model was used as a frozen feature extractor to generate contextual tweet embeddings. To process the full dataset efficiently, embeddings were computed in chunks and clustered using MiniBatchKMeans with five clusters. Incremental PCA was applied to project the high-dimensional embedding space into two dimensions for visualization.

Because the final trend detection stage was unsupervised, the evaluation focused on cluster size, semantic interpretation, sentiment composition within clusters, and visual separability in the PCA projection.

The five-cluster solution distributed tweets across the dataset in a reasonably balanced way, although Clusters 0 and 4 were the most dominant. Cluster 0 was the largest semantic group and Cluster 3 was the smallest.

Cluster	Tweet Count	Share of Dataset (%)
0	407,997	25.76
1	347,365	21.94
2	230,326	14.54
3	201,107	12.70
4	396,776	25.06

Table 5.1. Distribution of tweets across the five semantic clusters.

4.9 Cluster Semantics and Thematic Interpretation

The top words indicate that the clusters capture broad conversational patterns rather than strict formal topics. They reflect recurring modes such as personal updates, casual online chat, supportive or appreciative expressions, socially expressive interaction, and work or event-related reporting.

Cluster 0. Top words include just, day, good, like, today, really. This cluster largely reflects general daily-life updates, reactions, and mild dissatisfaction or complaint-oriented posts.

Cluster 1. Top words include quot, just, lol, like, good, got. This cluster largely reflects casual conversational chatter, informal responses, and internet-style expressions.

Cluster 2. Top words include good, thanks, day, work, love, just. This cluster reflects a mixed set of everyday routines, appreciative expressions, and personal reactions rather than a single narrow topic.

Cluster 3. Top words include lol, quot, haha, love, good, just. This cluster mainly reflects socially interactive, emotionally expressive, and conversational posts.

Cluster 4. Top words include just, day, work, new, today, good. This cluster mainly reflects work-related updates, platform chatter, and day-to-day event reporting.

4.10 Sentiment Composition Within Clusters

Post-clustering sentiment analysis showed that each semantic cluster contained both positive and negative tweets, although one polarity slightly dominated within most clusters.

Cluster	Negative (%)	Positive (%)	Dominant Sentiment
0	57.25	42.75	Negative
1	53.60	46.40	Negative
2	45.14	54.86	Positive
3	42.78	57.22	Positive
4	46.51	53.49	Positive

Table 5.2. Sentiment distribution across semantic clusters.

Clusters 0 and 1 were negative-leaning, whereas Clusters 2, 3, and 4 were positive-leaning. Cluster 3 was the most positive cluster at 57.22%, while Cluster 0 was the most negative at 57.25%.

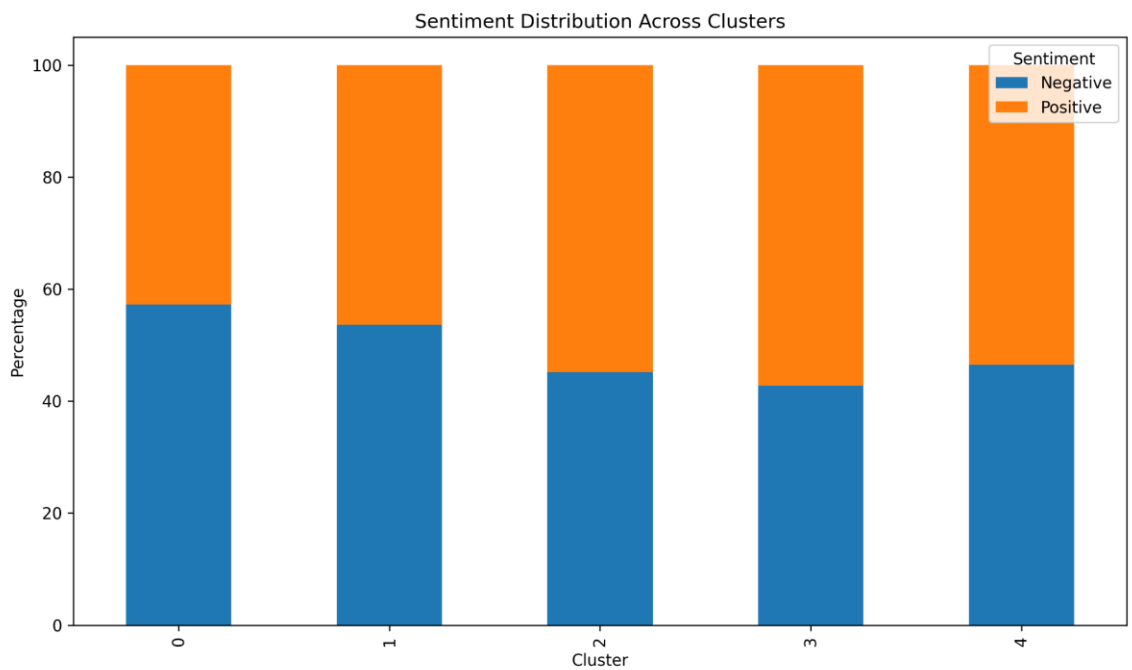

Figure 5.1. Percentage of positive and negative tweets within each semantic cluster.

4.11 PCA Visualization Analysis

The PCA projection reveals a meaningful overall cluster structure, particularly in the left-side separation of one cluster and the upper-right and lower-right concentration of others. However, visible overlap remains among several groups.

This overlap is expected because clustering was performed in the original high-dimensional transformer embedding space, while PCA compresses that space into only two components. Social media text is also short, noisy, and semantically mixed, so partial overlap does not imply model failure.

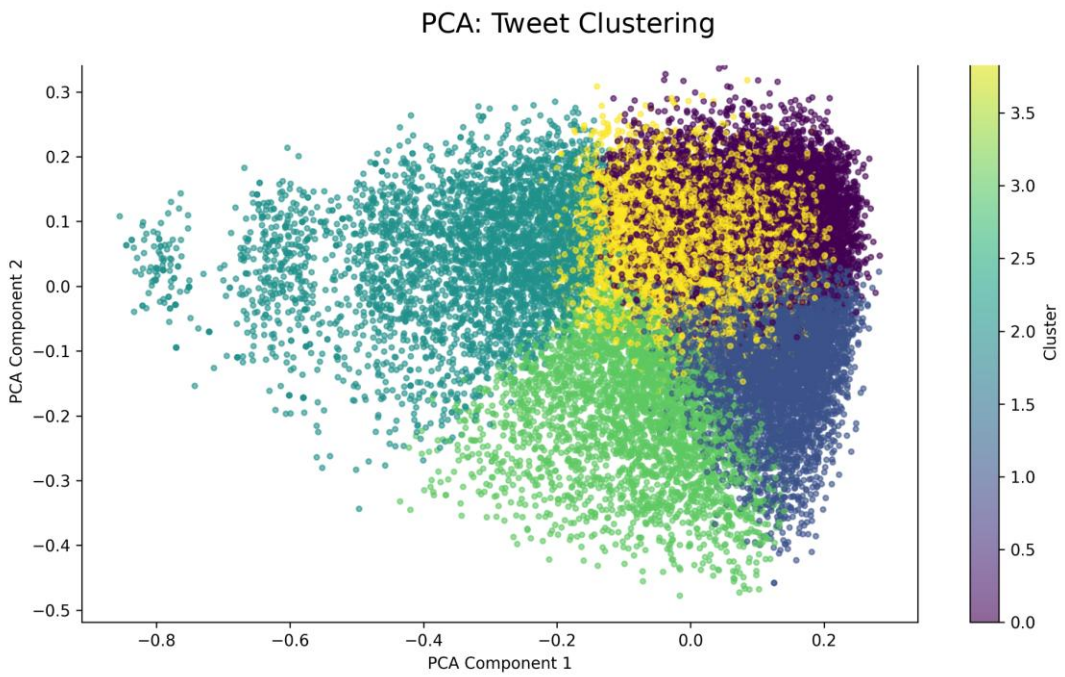

Figure 5.2. PCA: Tweet Clustering.

4.12 Discussion

The results show that transformer embeddings combined with scalable chunked processing can support semantic trend discovery on a very large social media corpus. The pipeline processed more than 1.58 million tweets without requiring full transformer fine-tuning.

The clusters should be interpreted as semantic usage patterns rather than strict sentiment classes. Each group retains a mixture of positive and negative tweets, which is useful for realistic trend monitoring because online discussions often combine topic similarity with mixed emotional tone.

A limitation of the current approach is that the final modeling stage is unsupervised; therefore, the study emphasizes cluster coherence, composition, and interpretability rather than supervised accuracy metrics.

4.13 Chapter Summary

This chapter presented the research design and methodology adopted for large-scale social media sentiment and trend analysis. The project was structured as a scalable analytics workflow using Apache Spark for preprocessing, transformer-based embeddings for semantic representation, and unsupervised clustering for trend discovery.

Key Findings

The methodology established a complete pipeline for handling a large social media dataset efficiently. Spark-based preprocessing enabled cleaning, label mapping, timestamp formatting, and extraction of useful temporal features across the corpus. DistilBERT embeddings provided contextual semantic representations of tweets, while MiniBatchKMeans supported scalable clustering on the full dataset through chunk-based processing. The workflow also produced multiple analytical outputs, including sentiment distributions, temporal patterns, semantic clusters, and PCA-based visualizations.

Key Interpretations

The methodology demonstrated that the system was able to meet the project's expectations in terms of scalability, interpretability, and practical implementation. The combination of distributed preprocessing and transformer-based semantic modeling was appropriate for short, noisy social media text. In addition, the use of unsupervised clustering allowed the project to move beyond simple sentiment counts and identify broader conversational patterns in the dataset. Overall, the chapter showed that the selected tools and design decisions were suitable for large-scale exploratory social media analysis.

5. Conclusion, Limitations and Future Work

This project demonstrated a scalable framework for social media sentiment and trend analysis by combining distributed preprocessing with transformer-based semantic modeling. Spark prepared the dataset, DistilBERT embeddings captured tweet meaning, and MiniBatchKMeans grouped the full corpus into five semantic clusters.

The results showed a clear semantic structure across the dataset, with both negative-leaning and positive-leaning clusters. Cluster 0 emerged as the most negative cluster and Cluster 3 as the most positive. The PCA visualization further confirmed that the embedding space preserved useful organization even though overlap remained after dimensionality reduction.

Overall, the project achieved its objective of building a practical big data analytics workflow for large-scale social media insight generation.

The project used an offline benchmark dataset rather than a fully deployed live social media stream, so the real-time architecture was demonstrated conceptually rather than integrated end to end. In addition, the transformer was used as a frozen feature extractor to keep computation tractable.

Future work can extend the system with live ingestion through Kafka or APIs, dynamic online clustering, latency benchmarking under streaming load, and transformer fine-tuning for supervised sentiment classification.

6. Appendix A.

A.1 Spark Data Loading and Schema Assignment

```python
def load_csv(
        self,
        file_path: str,
        header: bool = True,
        infer_schema: bool = True,
        multiline: bool = True,
        escape: str = '"',
    ) -> DataFrame:
        """
        Load a CSV file into a Spark DataFrame.

        Args:
            File Path (str): Path to the CSV File
            Header (bool): Include the headers from the CSV File
            Infer Schema (bool): Infer the Schema from the CSV File
            Multiline (bool): Allow Multiple lines in the dataset
            Escape (str): String Patterns that are to be Ignored

        Returns:
            (DataFrame): DataFrame created through PySpark object
        """
        spark = self.create_session()

        df = (
            spark.read
            .option("header", str(header).lower())
            .option("inferSchema", str(infer_schema).lower())
            .option("multiline", str(multiline).lower())
            .option("escape", escape)
            .csv(file_path)
        )
        return df
```

A.2 Spark Preprocessing Pipeline

```python
    def select_columns(self, columns: list[str]) -> "SparkPreprocessor":
        self.df = self.df.select(*columns)
        return self

    def drop_nulls(self) -> "SparkPreprocessor":
        required_columns = [self.text_column]
        if self.timestamp_column:
```

```
            required_columns.append(self.timestamp_column)

        self.df = self.df.dropna(subset=required_columns)
        return self

    def drop_duplicates(self) -> "SparkPreprocessor":
        self.df = self.df.dropDuplicates()
        return self

    def clean_text(self) -> "SparkPreprocessor":
        """
        Light cleaning only.
        Keeps transformer-friendly structure while removing obvious noise.
        """
        cleaned_col = F.col(self.text_column)

        cleaned_col = F.regexp_replace(cleaned_col, r"http\S+|www\S+", "")
        cleaned_col = F.regexp_replace(cleaned_col, r"@\w+", "")
        cleaned_col = F.regexp_replace(cleaned_col, r"#", "")
        cleaned_col = F.regexp_replace(cleaned_col, r"[^a-zA-Z0-9\s]", " ")
        cleaned_col = F.regexp_replace(cleaned_col, r"\s+", " ")
        cleaned_col = F.trim(cleaned_col)

        self.df = self.df.withColumn(self.text_column, cleaned_col)
        return self

    def map_target_labels(self) -> "SparkPreprocessor":
        self.df = self.df.withColumn(
            "target",
            F.when(F.col("target") == 4, 1).otherwise(0)
        )
        return self

    def remove_empty_text(self) -> "SparkPreprocessor":
        self.df = self.df.filter(F.length(F.col(self.text_column)) > 0)
        return self

    def format_timestamp(self, input_format: str | None = None) ->
"SparkPreprocessor":
        """
        Convert timestamp column to Spark timestamp type if available.
        """

        if self.timestamp_column:
            # Remove weekday prefix like "Mon "
            cleaned_ts = F.regexp_replace(F.col(self.timestamp_column),
r"^[A-Za-z]{3}\s+", "")

            if input_format:
```

```
                self.df = self.df.withColumn(
                    self.timestamp_column,
                    F.to_timestamp(cleaned_ts, input_format),
                )
            else:
                self.df = self.df.withColumn(
                    self.timestamp_column,
                    F.to_timestamp(cleaned_ts),
                )

        return self

    def add_time_features(self) -> "SparkPreprocessor":
        """
        Add useful time-based columns for trend analysis.
        """
        if self.timestamp_column:
            self.df = (
                self.df
                .withColumn("year", F.year(F.col(self.timestamp_column)))
                .withColumn("month", F.month(F.col(self.timestamp_column)))
                .withColumn("day",
F.dayofmonth(F.col(self.timestamp_column)))
                .withColumn("hour", F.hour(F.col(self.timestamp_column)))
            )
        return self
```

A.3 Exploratory Data Analysis and Visualization

```
    def select_columns(self, columns: list[str]) -> "SparkPreprocessor":
        self.df = self.df.select(*columns)
        return self

    def drop_nulls(self) -> "SparkPreprocessor":
        required_columns = [self.text_column]
        if self.timestamp_column:
            required_columns.append(self.timestamp_column)

        self.df = self.df.dropna(subset=required_columns)
        return self

    def drop_duplicates(self) -> "SparkPreprocessor":
```

```
        self.df = self.df.dropDuplicates()
        return self

    def clean_text(self) -> "SparkPreprocessor":
        """
        Light cleaning only.
        Keeps transformer-friendly structure while removing obvious noise.
        """
        cleaned_col = F.col(self.text_column)

        cleaned_col = F.regexp_replace(cleaned_col, r"http\S+|www\S+", "")
        cleaned_col = F.regexp_replace(cleaned_col, r"@\w+", "")
        cleaned_col = F.regexp_replace(cleaned_col, r"#", "")
        cleaned_col = F.regexp_replace(cleaned_col, r"[^a-zA-Z0-9\s]", "
")
        cleaned_col = F.regexp_replace(cleaned_col, r"\s+", " ")
        cleaned_col = F.trim(cleaned_col)

        self.df = self.df.withColumn(self.text_column, cleaned_col)
        return self

    def map_target_labels(self) -> "SparkPreprocessor":
        self.df = self.df.withColumn(
            "target",
            F.when(F.col("target") == 4, 1).otherwise(0)
        )
        return self

    def remove_empty_text(self) -> "SparkPreprocessor":
        self.df = self.df.filter(F.length(F.col(self.text_column)) > 0)
        return self

    def format_timestamp(self, input_format: str | None = None) ->
"SparkPreprocessor":
        """
        Convert timestamp column to Spark timestamp type if available.
        """

        if self.timestamp_column:
            # Remove weekday prefix like "Mon "
            cleaned_ts = F.regexp_replace(F.col(self.timestamp_column),
r"^[A-Za-z]{3}\s+", "")

            if input_format:
                self.df = self.df.withColumn(
                    self.timestamp_column,
                    F.to_timestamp(cleaned_ts, input_format),
```

```
                )
            else:
                self.df = self.df.withColumn(
                    self.timestamp_column,
                    F.to_timestamp(cleaned_ts),
                )

        return self

    def add_time_features(self) -> "SparkPreprocessor":
        """
        Add useful time-based columns for trend analysis.
        """
        if self.timestamp_column:
            self.df = (
                self.df
                .withColumn("year", F.year(F.col(self.timestamp_column)))
                .withColumn("month",
F.month(F.col(self.timestamp_column)))
                .withColumn("day",
F.dayofmonth(F.col(self.timestamp_column)))
                .withColumn("hour", F.hour(F.col(self.timestamp_column)))
            )
        return self
```

A.3 Exploratory Data Analysis and Visualization

```
from pathlib import Path
import matplotlib.pyplot as plt
from pandas import DataFrame
import pyspark
from typing import List, Any


class DataVisualization:
    """
    This Module Runs Visualization on the preprocess dataframe and then
save
    the results in the directory provided.
    """

    def __init__(self, /, outDir: Path | str | None = None) -> None:
        if outDir is not None:
            self.outDir: Path = Path(outDir)
```

```
        else:
            self.outDir: Path = Path("visual_ref")

        self.outDir.mkdir(exist_ok=True)

    def __sentiment_Distribution__(self, /, sp_df: pyspark.sql.DataFrame)
-> None:
        df: DataFrame = sp_df.toPandas()

        plt.figure(figsize=(8, 5))
        plt.bar(df["target"], df["tweet_count"])
        plt.xticks([0, 1], ["Negative", "Positive"])
        plt.title("Sentiment Distribution")
        plt.xlabel("Sentiment")
        plt.ylabel("Tweet Count")
        plt.savefig(self.outDir / "sentiment_distribution.png",
bbox_inches="tight")
        plt.show()

    def __tweet_per_hour__(self, /, sp_df: pyspark.sql.DataFrame) -> None:
        df: DataFrame = sp_df.toPandas()

        plt.figure(figsize=(10, 5))
        plt.plot(df["hour"], df["tweet_count"], marker="o")
        plt.title("Tweets Per Hour")
        plt.xlabel("Hour")
        plt.ylabel("Tweet Count")
        plt.xticks(range(0, 24))
        plt.savefig(self.outDir / "tweets_per_hour.png",
bbox_inches="tight")
        plt.show()

    def __tweet_per_day__(self, /, sp_df: pyspark.sql.DataFrame) -> None:
        daily_pd: DataFrame = sp_df.toPandas()

        # create a readable date label
        daily_pd["date_label"] = (
            daily_pd["year"].astype(str) + "-" +
            daily_pd["month"].astype(str).str.zfill(2) + "-" +
            daily_pd["day"].astype(str).str.zfill(2)
        )

        plt.figure(figsize=(12, 5))
        plt.plot(daily_pd["date_label"], daily_pd["tweet_count"],
marker="o")
        plt.title("Tweets Per Day")
        plt.xlabel("Date")
```

```
        plt.ylabel("Tweet Count")
        plt.xticks(rotation=45)
        plt.savefig(self.outDir / "tweets_per_day.png",
bbox_inches="tight")
        plt.show()

    def __sentiment_over_time__(self, /, sp_df: pyspark.sql.DataFrame) ->
None:
        df: DataFrame = sp_df.toPandas()

        df["date_label"] = (
            df["year"].astype(str) + "-" +
            df["month"].astype(str).str.zfill(2) + "-" +
            df["day"].astype(str).str.zfill(2)
        )

        pivot_df = df.pivot_table(
            index="date_label",
            columns="target",
            values="tweet_count",
            fill_value=0
        )

        plt.figure(figsize=(12, 5))
        pivot_df.plot(ax=plt.gca(), marker="o")
        plt.title("Sentiment Trend Over Time")
        plt.xlabel("Date")
        plt.ylabel("Tweet Count")
        plt.xticks(rotation=45)
        plt.savefig(self.outDir / "sentiment_trend_over_time.png",
bbox_inches="tight")
        plt.show()
```

A.4 Transformer Embedding Generation

```
def list_csv_parts(input_dir: str) -> list[str]:
    files = sorted(glob.glob(os.path.join(input_dir, "*.csv")))
    if not files:
        raise FileNotFoundError(f"No CSV files found in {input_dir}")
    return files


def iter_chunks(input_dir: str, chunksize: int):
    for file_path in list_csv_parts(input_dir):
        for chunk in pd.read_csv(file_path, usecols=USECOLS,
chunksize=chunksize):
```

```
            yield chunk


def clean_chunk(df: pd.DataFrame) -> pd.DataFrame:
    df = df.copy()
    df[TEXT_COL] = df[TEXT_COL].astype(str).str.replace(r"\s+", " ",
regex=True).str.strip()
    df = df[df[TEXT_COL] != ""]
    df = df.drop_duplicates(subset=[TEXT_COL])
    df = df.reset_index(drop=True)
    return df


def mean_pooling(last_hidden_state: torch.Tensor, attention_mask:
torch.Tensor) -> torch.Tensor:
    mask = attention_mask.unsqueeze(-
1).expand(last_hidden_state.size()).float()
    masked_embeddings = last_hidden_state * mask
    summed = torch.sum(masked_embeddings, dim=1)
    counts = torch.clamp(mask.sum(dim=1), min=1e-9)
    return summed / counts


def embed_texts(texts: list[str], tokenizer, model, device, batch_size: int,
max_len: int) -> np.ndarray:
    all_embeddings = []

    with torch.no_grad():
        for i in range(0, len(texts), batch_size):
            batch_texts = texts[i:i + batch_size]

            encoded = tokenizer(
                batch_texts,
                padding=True,
                truncation=True,
                max_length=max_len,
                return_tensors="pt"
            )

            encoded = {k: v.to(device) for k, v in encoded.items()}
            outputs = model(**encoded)

            pooled = mean_pooling(outputs.last_hidden_state,
encoded["attention_mask"])
            pooled = pooled.cpu().numpy()
            all_embeddings.append(pooled)

    embeddings = np.vstack(all_embeddings)
    embeddings = normalize(embeddings)
```

```
    return embeddings


def tokenize_for_keywords(text: str) -> list[str]:
    tokens = re.findall(r"[a-zA-Z]{3,}", text.lower())
    return [t for t in tokens if t not in ENGLISH_STOP_WORDS]


# ========================
# MODEL SETUP
# ========================
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device:", device)

tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
model = AutoModel.from_pretrained(MODEL_NAME)
model.to(device)
model.eval()

kmeans = MiniBatchKMeans(
    n_clusters=N_CLUSTERS,
    random_state=RANDOM_STATE,
    batch_size=KMEANS_BATCH_SIZE,
    n_init="auto"
)

ipca = IncrementalPCA(n_components=PCA_COMPONENTS)
```

A.5 Clustering and PCA Pipeline

```
total_rows = 0
chunk_count = 0

for chunk in iter_chunks(INPUT_DIR, CSV_CHUNKSIZE):
    chunk = clean_chunk(chunk)
    if chunk.empty:
        continue

    texts = chunk[TEXT_COL].tolist()
    embeddings = embed_texts(
        texts=texts,
        tokenizer=tokenizer,
        model=model,
        device=device,
        batch_size=EMBED_BATCH_SIZE,
        max_len=MAX_LEN
    )
```

```
    kmeans.partial_fit(embeddings)

    if embeddings.shape[0] >= PCA_COMPONENTS:
        ipca.partial_fit(embeddings)

    total_rows += len(chunk)
    chunk_count += 1

    if chunk_count % 10 == 0:
        print(f"Processed chunks: {chunk_count}, rows so far: {total_rows}")
```

A.6 Cluster-wise Sentiment Analysis

```
df = pd.read_csv(INPUT_FILE)

df["target_label"] = df["target"].map({
    0: "Negative",
    1: "Positive"
})

# =========================
# COUNTS
# =========================
cluster_target_counts = (
    df.groupby(["cluster", "target", "target_label"])
    .size()
    .reset_index(name="tweet_count")
    .sort_values(["cluster", "target"])
    .reset_index(drop=True)
)

cluster_target_counts.to_csv(
    os.path.join(OUTPUT_DIR, "cluster_target_counts.csv"),
    index=False
)

# =========================
# PERCENTAGES
# =========================
cluster_target_percentages = cluster_target_counts.copy()

cluster_target_percentages["percentage"] = (
    cluster_target_percentages["tweet_count"]
    /
cluster_target_percentages.groupby("cluster")["tweet_count"].transform("sum")
) * 100
```

```python
cluster_target_percentages.to_csv(
    os.path.join(OUTPUT_DIR, "cluster_target_percentages.csv"),
    index=False
)

# ========================
# DOMINANT SENTIMENT
# ========================
dominant_sentiment = (
    cluster_target_percentages.loc[
        cluster_target_percentages.groupby("cluster")["percentage"].idxmax()
    ]
    .sort_values("cluster")
    .reset_index(drop=True)
)

dominant_sentiment.to_csv(
    os.path.join(OUTPUT_DIR, "dominant_sentiment_per_cluster.csv"),
    index=False
)

# ========================
# PIVOTS
# ========================
pivot_counts = cluster_target_counts.pivot_table(
    index="cluster",
    columns="target_label",
    values="tweet_count",
    fill_value=0
)

pivot_percentages = cluster_target_percentages.pivot_table(
    index="cluster",
    columns="target_label",
    values="percentage",
    fill_value=0
)

pivot_counts.to_csv(os.path.join(OUTPUT_DIR,
"cluster_target_counts_pivot.csv"))
pivot_percentages.to_csv(os.path.join(OUTPUT_DIR,
"cluster_target_percentages_pivot.csv"))

# ========================
# PLOT
# ========================
plt.figure(figsize=(10, 6))
pivot_percentages.plot(kind="bar", stacked=True, ax=plt.gca())
```

```
plt.title("Sentiment Distribution Across Clusters")
plt.xlabel("Cluster")
plt.ylabel("Percentage")
plt.legend(title="Sentiment")
plt.tight_layout()
plt.savefig(os.path.join(OUTPUT_DIR, "cluster_sentiment_distribution.png"),
dpi=300)
plt.show()
```